

1 INTRODUCCION

Actualmente, la universidad recibe y atiende solicitudes estudiantiles relacionadas con mantenimiento, soporte tecnológico, infraestructura, equipamiento y otros servicios institucionales mediante diferentes canales, entre ellos formularios físicos, grupos de mensajería y correo electrónico.

La utilización de estos canales de manera independiente genera dificultades en la gestión del proceso. Una misma solicitud puede ser comunicada por diferentes medios, dificultando determinar si corresponde a un requerimiento nuevo o a uno previamente registrado. Asimismo, cuando una solicitud es derivada entre diferentes responsables, puede resultar difícil conocer su situación actual, quién está encargado de atenderla y qué acciones se realizaron anteriormente.

Para los estudiantes, esta situación puede generar incertidumbre respecto al estado de sus solicitudes y la necesidad de realizar consultas adicionales para conocer su avance.

Para el personal administrativo, la información distribuida entre diferentes canales dificulta la priorización de solicitudes, la asignación de responsables y el seguimiento de los requerimientos pendientes.

Además, determinados reportes requieren información adicional para que el personal encargado pueda evaluar adecuadamente la situación. Por ejemplo, un problema relacionado con infraestructura o equipamiento puede ser descrito de manera diferente por cada estudiante, dificultando la interpretación del requerimiento cuando no se dispone de evidencia complementaria.

La generación de información consolidada también representa una dificultad para la gestión administrativa, debido a que resulta necesario recopilar información de diferentes medios para determinar aspectos como:

- Cantidad de solicitudes recibidas.
- Solicitudes pendientes de atención.
- Solicitudes según tipo.
- Solicitudes según prioridad.
- Tiempo de atención.
- Responsables asignados.
- Solicitudes cerradas.

Ante esta situación, la universidad requiere una solución que centralice la gestión de solicitudes y permita registrar, atender y consultar la información asociada a cada requerimiento durante todo su ciclo de vida.

La solución propuesta es CAMPUS CONNECT, una plataforma integrada mediante una aplicación Web, una API REST y una aplicación móvil.

2 OBJETIVO

Desarrollar una plataforma que permita gestionar solicitudes universitarias y recursos institucionales, proporcionando seguimiento, trazabilidad y comunicación entre estudiantes y administrativos.

3 RESTRICCIONES

Backend

- Laravel 12, .NET Core

Base de Datos

- PostgreSQL
- SQL Server

Repositorio

- GitHub

Aplicación Web

- Blade
- Razor views

Aplicación Móvil

- Flutter
- .NET MAUI

1. ORGANIZACIÓN DE EQUIPO

Cada equipo estará compuesto por:

Desarrollador A

Responsable principal de:

- Frontend Web
- Dashboard Administrativo
- Reportes
- Gestión de Recursos

Desarrollador B

Responsable principal de:

- Aplicación Móvil
- Consumo de API
- Registro de Solicitudes
- Seguimiento de Solicitudes

Responsabilidad Compartida

Ambos deben participar en:

- Diseño de base de datos.
- Desarrollo de API.
- Resolución de conflictos Git.
- Pull Requests.
- Integración Final.

La solución no será considerada terminada si no cuenta con pruebas.

Desarrollador A

Frontend Web

Responsable de:

- Pruebas de consumo de API.
- Validación de formularios móviles.
- Manejo de errores.
- Verificación de autenticación.
- Registro de evidencias.
- Pruebas mínimas
- Inicio de sesión.
- Crear solicitud.
- Adjuntar evidencia.
- Consultar seguimiento.
- Visualizar comentarios.

Desarrollador B

Aplicación Móvil

Responsable de:

- Pruebas funcionales de los formularios.
- Validación de reglas de negocio.
- Validación de navegación.
- Verificación de reportes.
- Registro de evidencias.
- Pruebas mínimas
- Crear solicitud.
- Editar solicitud.
- Eliminar solicitud.
- Cambiar estado.
- Asignar responsable.
- Generar reporte.

4 HERRAMIENTAS DE PRUEBAS

Pruebas Unitarias

Dependiendo de la tecnología utilizada deben implementar pruebas con:

Laravel: Php Unit o Test.

.Net Core: xUnit o Nunit

Pruebas de Integración

Deben demostrar que:

- Web API Base de Datos
- Movil, API y Base de Datos funcionan correctamente

5 INTEGRACIÓN CONTÍNUA

Además, deben demostrar:

Ejecución de pruebas antes del Merge

Ningún Pull Request podrá ser aceptado si:

- No compila.
- No ejecuta pruebas.
- Tiene errores funcionales.

6 EVIDENCIAS

Los estudiantes deben presentar:

- 5 pruebas mínimamente.
- Diseño de casos de uso.
- Arquitectura
- Cumplimiento de funcionalidades
- Repositorio en GitHub.